

CEG3585 - INTRODUCTION AUX RÉSEAUX D'ORDINATEURS

TUTORIEL 4 - LES SOCKETS

Architecture TCP/IP

- Les applications sont développées avec le socket API (en Java les classes Socket et ServerSocket)
- Adresse socket:
 - Port TCP
 - Adresse IP
 - Les deux adresses sockets de chaque bout d'une connexion servent à identifier la connexion

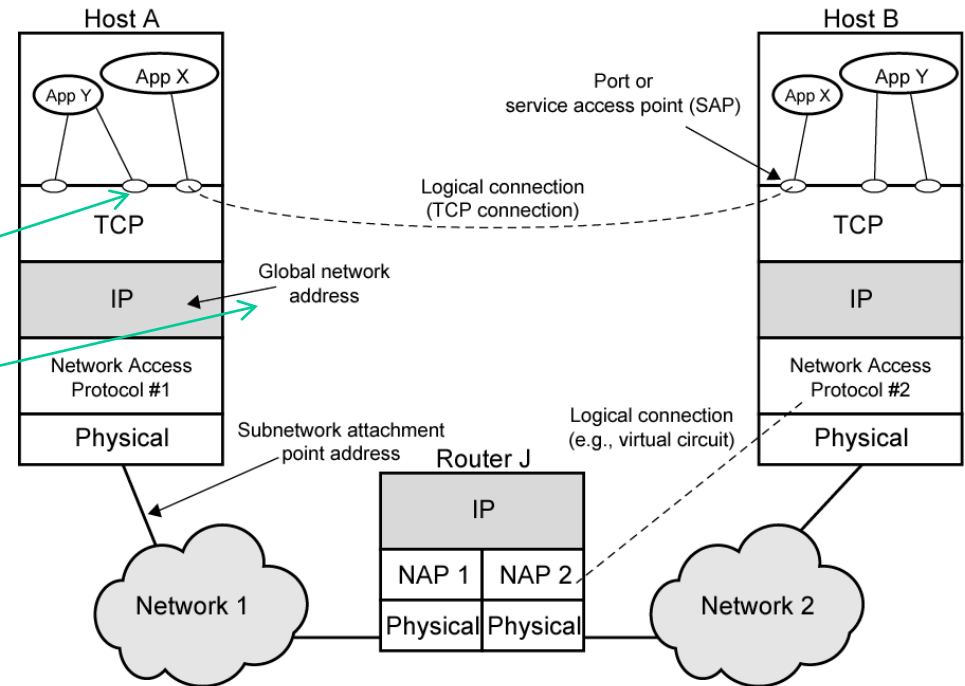


Figure 18.2 TCP/IP Concepts

La programmation "socket"

- Les sockets sont des interfaces qui peuvent communiquer entre elles en s'interconnectant à travers un réseau.
- Une communication de réseau peut se réaliser en échangeant des données de messages transmis entre des sockets.
- Les messages sont mis en queue au niveau de la socket transmetteur jusqu'à ce que le protocole du réseau les expédie. A leur arrivée, les messages sont mis en file au niveau de la socket réceptrice jusqu'à ce que processus de réception les traite.

La programmation socket

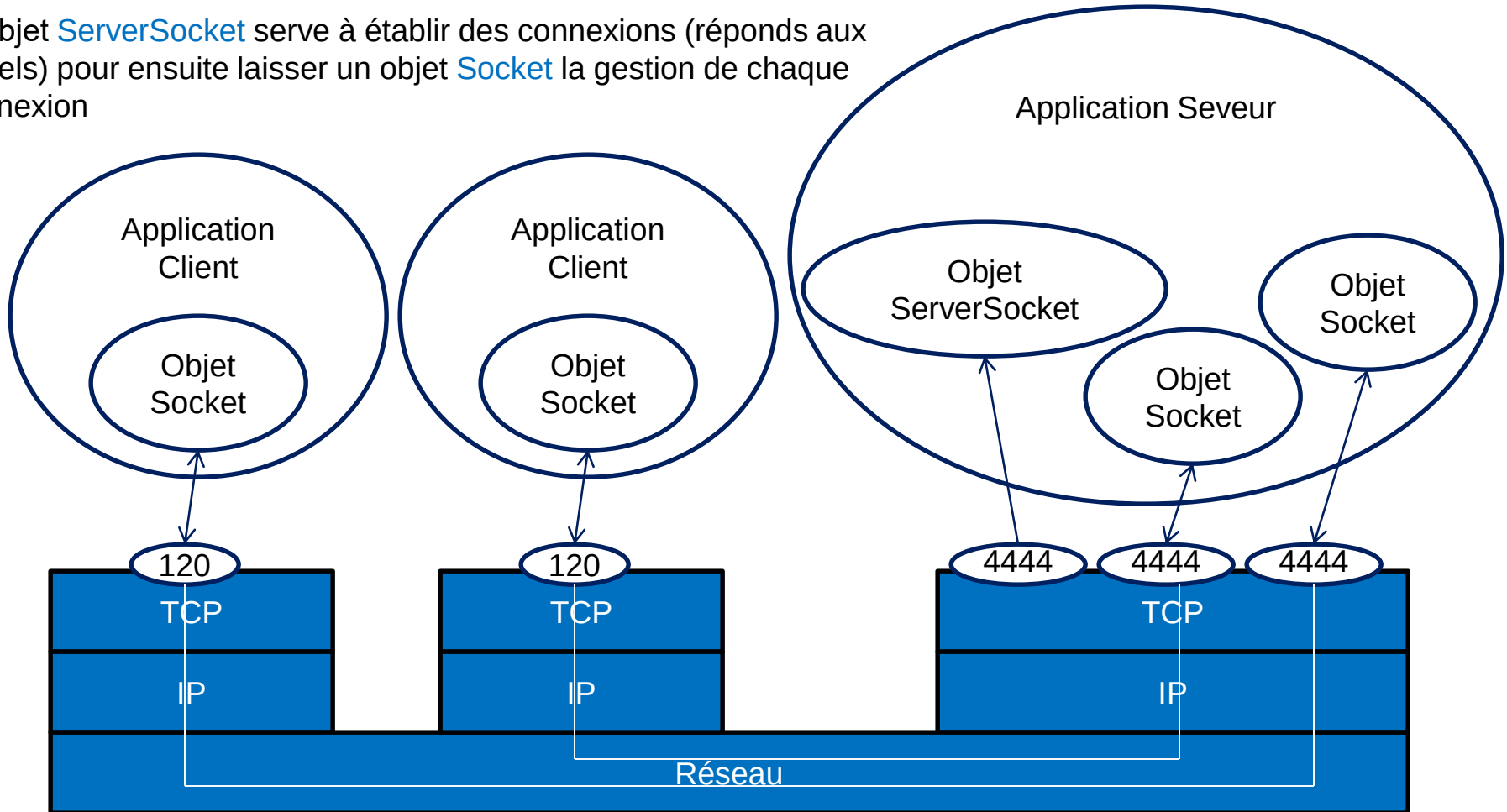
- Une socket Internet est identifiée par le système opératoire comme une combinaison unique de :
 - Protocole (TCP, UDP ou IP à l'état pur)
 - Adresse socket locale
 - Une adresse local IP
 - Un numéro de port local
 - Adresse socket du nœud distant (Uniquement pour de socket TCP établies)
 - Une adresse éloignée IP
 - Un numéro de port éloignée
 - Donc dans le cas du TCP, les deux adresses sockets

La programmation socket

- Modèle Client-Serveur
 - Un processus de logiciel client peut initier une session de communication pendant que le serveur attend des requêtes de clients.
 - La plupart des applications d'entreprises écrites aujourd'hui utilisent le modèle Client-Serveur. Il en est de même pour les protocoles d'applications principales de l'internet tels que HTTP, SMTP, Telnet, DNS, etc.

Modèle client-serveur avec sockets en Java

*L'objet `ServerSocket` serve à établir des connexions (réponds aux appels) pour ensuite laisser un objet `Socket` la gestion de chaque connexion



La programmation socket

- Les commandes (*Open-Read-Write-Close*) :
 - Avant que le processus d'un usager puisse réaliser des opérations de réseautage, il lui faut déclarer une socket pour spécifier et obtenir les permissions de communication du réseau.
 - Une fois la socket créée, le processus de l'usager doit faire appel à d'autres commandes (Lire ou Ecrire) pour échanger des données.
 - Une fois toutes les opérations de transfert de données complétées, le processus de l'usager doit utiliser la commande (Close) pour informer le système opératoire de la fin de la communication .

Java Socket – Connexion (Open)

- **Client**

- La classe Socket
 - Cette classe réalise une socket client (appelée "socket").

```
public Socket(String adresse, int port)  
    throws IOException
```

- adresse: - l'adresse IP (ou nom d'hôte).
- port – le numéro de port du serveur.
- Le port local sera déterminé par le system d'exploitation.

Java Socket – Connexion

- Exemple

```
Socket monClient;  
try {  
    monClient = new Socket("nom",  
                           numéroPort);  
}    // nom: nom d'hôte ou adresse IP  
catch (IOException e) {  
    System.out.println(e);  
}
```

Java Socket – Connexion (Open)

- **Serveur**

- La classe `ServerSocket`
 - Cette classe réalise une socket serveur. Une socket serveur attend des requêtes (telle que la demande de connexion TCP) provenant du réseau. Elle réalise des opérations, en fonction de la requête, et possiblement retourne un résultat au demandeur.

```
public ServerSocket(int port)
    throws IOException
```

- port - le numéro du port du serveur pour écouter et accepter les appels entrants..

```
public Socket accept()
    throws IOException
```

- Méthode de la classe `ServerSocket`: retourne la nouvelle `Socket` pour gérer la connexion.

Java Socket – Connexion

- Exemple

- Du côté du serveur:
 - La création du socket serveur :

```
ServerSocket monService;  
try {  
    monService = new ServerSocket(portNumber);  
}  
catch (IOException e) {  
    System.out.println(e);  
}
```

- L'écoute du réseau se fait avec accept()

```
Socket serviceSocket = null;  
try {  
    serviceSocket = monService.accept();  
}  
catch (IOException e) {  
    System.out.println(e);  
}
```

Socket java : Lecture

class **BufferedReader**

- Cette classe permet une lecture d'un String Java.
- Méthode de la classe Socket: retourne une référence à un objet InputStream pour la lecture du socket.

```
public InputStream getInputStream()  
    throws IOException
```

- L'objet retourné par devrait être encapsulé dans un objet BufferedReader qui offre un tampon de données, où le Reader réfère à l'objet InputStream.

```
public BufferedReader(Reader in)
```

- Méthode de la classe BufferedReader: retourne la ligne suivante du texte de la chaine d'entrées.

```
public final String readLine()  
    throws IOException
```

Java Socket - Lecture

Dans le client et le serveur:

```
BufferedReader input;
try {
    input = new BufferedReader(
        new InputStreamReader(
            mySocket.getInputStream()
        )
    )
}
catch (IOException e) {
    System.out.println(e);
}
String message = input.readLine(); // Bloque jusqu'à
                                     // l'arrivée de
données
```

Java Socket - Écriture

```
class PrintWriter(OutputStream out)
```

- Cette classe permet une écriture d'un String Java à un socket Java.

- `public OutputStream getOutputStream()`

throws IOException

- Returns un référence à un object OutputStream à envoyer au constructeur PrintWriter.

- `public final void println(String s)`

throws IOException

- Méthode de la classe PrintWriter pour écrire un String Java, S.
- Notez que le println est nécessaire afin que la méthode `readLine()` de la classe `BufferedReader` lise la ligne (la fin de ligne est élevé par `readLine()`).

Java Socket - Écrire

```
PrintWriter output;  
try {  
    output = new PrintWriter(  
        MyClient.getOutputStream() );  
}  
catch (IOException e) {  
    System.out.println(e);  
}  
output.println("une chaine de caractères");
```

Java Socket – Fermeture (Close)

- `public void close()`
 throws `IOException`
- Une fois une socket fermée, elle n'est plus disponible à communiquer. Une nouvelle objet `Socket` doit être créé. Si un connexion est associée à ce socket, il s'en suit qu'il est aussi fermé.
- Cette méthode est présent dans la classe `Socket` et la classe `ServerSocket`.
- Cette méthode est aussi présente dans les objets `BufferedReader` et `PrintWriter`.

Java Socket - Close

Du coté du client :

```
try {  
    output.close();  
    input.close();  
    MyClient.close();  
}  
catch (IOException e) { System.out.println(e); }
```

Du coté du server (pour chaque connexion, i.e. Socket, voir du côté du client):

```
try {  
    serviceSocket.close();  
    MyService.close();  
}  
catch (IOException e) {System.out.println(e);}
```